Machine Learning-based Response Time Prediction and Dynamic Load Balancing Optimization for Private Cloud

Yang Zhou
Digital-X AIoT Research Center
Pusan National University
Busan, Korea
zhouyang@pusan.ac.kr

Lingxiao Zhou
Digital-X AIoT Research Center
Pusan National University
Busan, Korea
zlx202488581@pusan.ac.kr

Taewoon Kim*
Digital-X AIoT Research Center
Pusan National University
Busan, Korea
taewoon@pusan.ac.kr

***Abstract*—To mitigate load imbalance, which may lead to increased response time in private cloud environments, we present a response time prediction approach utilizing an eXtreme Gradient Boosting (XGBoost) model, alongside three dynamic load-balancing optimization models. These methods are designed to optimize both response time and resource allocation. Experimental evaluations conducted on a small-scale testbed demonstrate that the XGBoost model achieves the lowest response time prediction error compared to the benchmark methods. Among the load-balancing approaches considered in this study, including the proposed optimal models, our response time minimization model, designated as Optimization Model 1, outperforms the others in terms of delay and resource utilization. Although the Double-DQN model exhibits greater adaptability to environmental dynamics, it incurs significantly higher computation time.**

***Index Terms*—Response time minimization, load balancing, optimization, machine learning, reinforcement learning.**

I. Introduction

Contemporary cloud computing environments are characterized by dynamic workload patterns and system behaviors, frequently causing load imbalances across private data center resources that result in both performance degradation such as response time and resource under/over-utilization. Accordingly, this study aims to develop response time prediction and load scheduling optimization techniques to enhance load balancing [1] efficiency, maximize resource utilization, and improve overall response time performance.

II. Proposed Idea

This study presents a response time prediction model alongside three dynamic load-balancing [1] optimization models, designed to enhance both the accuracy of load-balancing operations and response time performance.

* Corresponding author: Taewoon Kim

This work was partly supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP)-ITRC(Information Technology Research Center) grant funded by the Korea government(MSIT)(IITP-2025-RS-2023-00260098, 50%) and IITP-Artificial Intelligence Convergence Innovation Human Resources Development grant funded by the Korea government(MSIT)(IITP-2025-RS-2023-00254177, 50%).

A. Response Time Prediction

To model the non-linear relationship between task characteristics and response time, we employ XGBoost [2] which is one of the most outperforming gradient boosting algorithm. The model takes as input a random integer n ranging from 1 to 500,000 (with 6,920 training samples), representing individual task complexity. The output r_i represents the predicted response time for processing each task or user request.

B. Dynamic Load Balancing Optimization Model 1: Direct Optimization

Eq. (1) is designed to minimize the weighted sum of resource usage and response time. The decision variable x_i denotes the allocated portion of the given task to server i; b^{mem} and b^{hdd} is the maximum values of $(u_i^{mem}+m\cdot x_i)$ and $(u_i^{hdd}+h\cdot x_i)$, respectively. Here, u_i^{mem} and u_i^{hdd} denotes the current memory and hard disk usage at server i, respectively, while m and h denotes the memory and hard disk requirements for processing the given task. The U_i^{mem} and U_i^{hdd} are the corresponding resource capacity at server i. Normalizing functions are defined as $f(b^{mem}) = b^{mem}/U_i^{mem}$, $f(b^{hdd}) = b^{hdd}/U_i^{hdd}$, and $f(\sum_{i=1}^{N} r_i \cdot x_i) = \sum_{i=1}^{N} r_i \cdot x_i / U^{res}$, where U^{res} is the maximum response time on servers. The weight α, β, and γ is temporarily set to 1/3 each in this study.

$$\min_{x_i, b^{mem}, b^{hdd}} \alpha \cdot f(b^{mem}) + \beta \cdot f(b^{hdd}) + \gamma \cdot f\left(\sum_{i=1}^{N} r_i \cdot x_i\right) \quad (1)$$

$$\begin{aligned} s.t. \quad & \forall i = 1, ..., N: \quad \sum_i x_i = 1, \quad x_i \in \{0, 1\} \\ & \forall i: \quad u_i^{mem} + m \cdot x_i \leqslant U_i^{mem}, u_i^{hdd} + h \cdot x_i \leqslant U_i^{hdd} \\ & \forall i: \quad u_i^{mem} + m \cdot x_i \leqslant b^{mem}, u_i^{hdd} + h \cdot x_i \leqslant b^{hdd} \end{aligned}$$

C. Dynamic Load Balancing Optimization Model 2: Minimizing The Resource Usage Deviations

Eq. (2) is designed to minimize the weighted sum of the standard deviation of allocated MEM/HDD resource across servers and the response time. The m_1 and m_2 denotes the maximum standard deviation of allocated memory and hard

Authorized licensed use limited to: Pusan National University Library. Downloaded on October 23,2026 at 04:31:42 UTC from IEEE Xplore. Restrictions apply.
978-4-907626-55-6/25/$31.00 ©2025 IPSJ

disk usage across servers, respectively, while m_3 denotes the maximum response time on servers.

$$\min_{x_i} \frac{\alpha}{m_1} \cdot \sqrt{\frac{1}{N}\sum_{i=1}^{N}(u_i^{mem} + m \cdot x_i - \frac{\sum_{i=1}^{N} u_i^{mem} + m}{N})^2} + \frac{\beta}{m_2} \cdot \sqrt{\frac{1}{N}\sum_{i=1}^{N}(u_i^{hdd} + h \cdot x_i - \frac{\sum_{i=1}^{N} u_i^{hdd} + h}{N})^2} + \frac{\gamma}{m_3} \cdot \sum_{i=1}^{N} r_i \cdot x_i \quad (2)$$

$$s.t. \quad \forall i = 1, ..., N : \sum_i x_i = 1, \quad x_i \in \{0, 1\}$$
$$\forall i : \quad u_i^{mem} + m \cdot x_i \leqslant U_i^{mem}, u_i^{hdd} + h \cdot x_i \leqslant U_i^{hdd}$$

D. Dynamic Load Balancing Optimization Model 3: Double DQN-based Approach

We designed the reward function for the Double DQN [3] model as in Eq. (3) to guide the agent to achieve lower response time, and to enhance the level of fair resource allocation. In addition, to address distribution shifts problem caused by random tasks, we introduced a Wasserstein distance metric. When a significant shift is detected, the system dynamically increases the exploration rate ε according to $\min(\varepsilon_{\max}, \varepsilon_{t-1} + 0.1)$ and applies a mild penalty ($-\mathcal{D}_t \cdot 0.01$) in reward. Here, $\varepsilon_{\max}$ denotes the maximum exploration rate (set to 1.0).

$$r(s, a) = \alpha \cdot \frac{1}{\hat{r} + \epsilon} + \beta \cdot \frac{1}{\hat{m} + \epsilon} + \gamma \cdot \frac{1}{\hat{h} + \epsilon} - \mathcal{D}_t \cdot 0.01 \quad (3)$$

- α, β, γ: weights (temporarily set to 1/3 each in this study)
- $\hat{r}, \hat{m}, \hat{h}$: normalized values, defined as $\hat{r} = r_a / U^{res}$, $\hat{m} = (u_a^{mem} + m)/U_a^{mem}$, and $\hat{h} = (u_a^{hdd} + h)/U_a^{hdd}$
- ϵ: smoothing factor to avoid dividing by zero
- $\mathcal{D}_t$: Wasserstein distance at time t

III. Evaluation

A. Testbed Setup

For experimental validation, we constructed a Docker Swarm-based private cloud cluster comprising one Raspberry Pi 4B as the manager node and three Raspberry Pi 3B devices as server nodes, all interconnected on a local area network. To simulate workload, a client issued HTTP requests at 30 ms intervals, each containing a random integer n requiring computation of the sum of all prime numbers below n. The experiment consisted of 20 episodes with 20 tasks each, totaling 400 tasks.

B. Experimental Result

a) Response Time Prediction: With an average processing time of 5.103 seconds, XGBoost achieved the lowest MSE error of 0.00119, substantially outperforming support vector regression (MSE: 0.002457) and the linear, lasso, and ridge regression models (MSE: 0.2438 for all). These results demonstrate the superior prediction performance of XGBoost.

b) Dynamic Load Balancing Optimization Model: As illustrated in Fig. 1, all three optimization models exhibited higher fairness (Jain's fairness index approaching to 0.998) in memory allocation ($\forall h = 0$, thus it's ignored) with stable curves. Regarding response time, Optimization Model 1 consistently achieved the lowest values across all episodes compared to traditional static load balancing (Round Robin), indicating the superior scheduling strategy and response speed. However, Optimization Model 2 and Double DQN exhibited higher and more fluctuating response time, performing even below Round Robin.

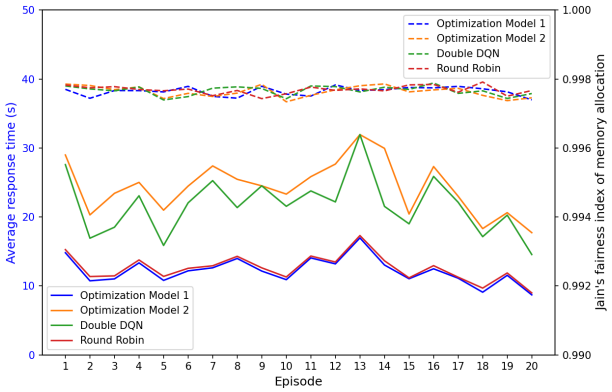


Fig. 1. Comparison of load balancing performance as to memory allocation and the average response time. The dotted line is Jain's fairness index (black y-axis), and the solid line is average response time (blue y-axis).

As shown in Fig. 2, Optimization Model 1 and 2 have similar execution time, while Double DQN is 13 to 14 times slower due to its complex structure, indicating lower execution efficiency despite higher adaptability.

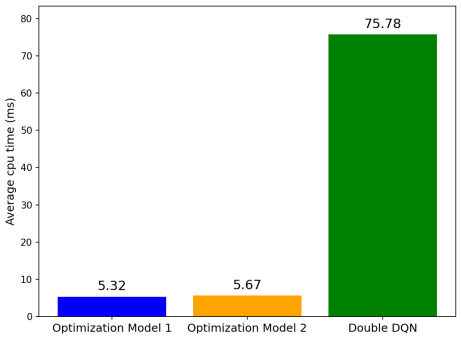


Fig. 2. Comparison of the cpu time for execution the proposed models.

References

[1] D. Kanellopoulos et al., "Dynamic Load Balancing Techniques in the IoT: A Review," *Symmetry*, vol. 14, no. 12, pp. 2554–2612, Dec. 2022.
[2] N. Zhai et al., "Multivariate time series forecast in industrial process based on XGBoost and GRU," in *Proc. IEEE 9th Joint Int. Inf. Technol. Artif. Intell. Conf. (ITAIC)*, IEEE, 2020, pp. 1-–6.
[3] H. v. Hasselt et al., "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, vol. 30, no. 1, pp. 2094–2100.

Authorized licensed use limited to: Pusan National University Library. Downloaded on October 23,2026 at 04:31:42 UTC from IEEE Xplore. Restrictions apply.